

BENNETT UNIVERSITY

School of Computer Science Engineering & Technology

MILESTONE 03

AI IN HEALTHCARE

Synchronization of Project Design & Implementation

DATA & BASELINE

MediGenius — AI-Powered Multi-Agent Medical Assistant

CSET343 | AI in Healthcare Mini Project

SUBMITTED BY

Nidhi Kuntal | E23CSEU2357

Krish Rajpurohit | E23CSEU2366

1. Design-to-Implementation Synchronization

MediGenius is implemented as a medical-information assistant, not an autonomous diagnostic system. Its repository separates pre-response safety rules, the LangGraph reasoning workflow, evidence retrieval, answer verification, and web/API delivery. This milestone maps those documented design elements to the actual modules and data flow. [1–3]

1.1 Executed request flow

A request reaches the FastAPI chat service. The safety router first detects crisis or emergency patterns; dosage/refusal checks and exact-match cache checks can also return without entering the graph. For remaining requests, the compiled LangGraph workflow performs memory recall, supervision, optional symptom structuring, planning, parallel retrieval or direct LLM response, synthesis, and diagnosis-claim verification. The service then applies dosage grounding, persists the exchange, and records an audit entry. [2–4]

Design responsibility	Implementation in repository
Pre-graph screening	core/safety_router.py; core/dosage_grounding.py; services/chat_service.py
Agent routing	core/langgraph_workflow.py; agents/medical_supervisor_agent.py; agents/planner.py
Evidence acquisition	agents/parallel_retrieval_agent.py; tools/pdf_loader.py; tools/vector_store.py
Answer generation and review	agents/executor.py; agents/diagnosis_verification_sub_agent.py
Persistence and delivery	services/database_service.py; tools/memory_store.py; api/v1/endpoints/; frontend/src/App.jsx

1.2 Routing and integration logic

The MedicalSupervisorAgent flags first-person symptom descriptions for SymptomAnalysisSubAgent; it does not diagnose. PlannerAgent selects retrieval for questions containing specified medical keywords and direct LLM processing otherwise. ParallelRetrievalAgent runs local RAG, Wikipedia, and Tavily concurrently with an 8-second per-branch timeout. A successful result is selected in the order curated medical PDF, Tavily, then Wikipedia; if no branch yields usable content, the graph routes to LLMAgent. [2, 5]

Implementation distinction: older retriever.py, wikipedia.py and tavily.py agent files remain in the repository but are not nodes in the active compiled graph. [1, 2]

2. Data Acquisition, Preparation & Retrieval Baseline

2.1 Document and knowledge sources

The configured local knowledge source is backend/data/medical_book.pdf (PDF_PATH). The backend startup checks for this file and, when present, processes it and initializes a persisted vector store. If the PDF is absent, initialization skips indexing and logs a warning. Wikipedia and Tavily are separate online retrieval branches; the system is not documented as using an annotated patient dataset for supervised diagnosis training. [1, 3, 6]

Data component	Code-level implementation
PDF ingestion	PyPDFLoader loads document pages; processing is invoked from backend/app/main.py.
Text segmentation	RecursiveCharacterTextSplitter.from_tiktoken_encoder; chunk_size=512, chunk_overlap=128; separators: paragraph, sentence, newline, space.
Dense embeddings	HuggingFaceEmbeddings with sentence-transformers/all-MiniLM-L6-v2.
Vector index	Persisted Chroma store with cosine similarity configuration (hnsw:space = cosine).
Query-time retrieval	get_retriever(k=3); short retrieved passages (≤50 characters) are excluded by the parallel retrieval agent.
Conversation data	SQLite persists messages; a separate Chroma collection, conversation_memory, stores session-filtered exchanges.

2.2 Data-to-response synchronization

The code links document loading to retrieval as follows: PDF pages → overlapping text chunks → MiniLM sentence embeddings → Chroma similarity search (top three) → candidate passages for response synthesis. In parallel, Tavily and Wikipedia may supply material. The executor uses at most the first three documents and takes at most 1,000 characters per document when assembling its evidence prompt. External text is sanitized for instruction-like strings before model use. [3, 5, 6, 7]

2.3 What “baseline” means in this repository

The deployed baseline is a retrieval-and-generation workflow, not a trained classifier with a reported accuracy score. The repository contains research notebooks, but the active inference path uses a pre-trained sentence-embedding model plus Groq-hosted language models via LiteLLM. The repository does not present a held-out clinical dataset, diagnostic accuracy, precision, recall, or F1 as results for this deployed assistant. [1, 2, 6, 8]

Data caveat: the repository documents the configured PDF and ingestion pipeline, but does not provide a versioned data dictionary, patient-label scheme, or quantified evaluation dataset in its README. [1]

3. Documented Baseline Behavior & Status

3.1 Repository-reported latency snapshot

The README reports the following single-run measurements against a live Groq-backed pipeline. They are implementation observations, not averages, percentiles, repeatability estimates, or clinical-quality measurements. [1]

Request path	Reported time	Interpretation
Crisis / emergency block	~0.01–0.02 s	Pre-graph static response
Paediatric / pregnancy dosing refusal	~0.01 s	Static refusal before retrieval
Drug-interaction refusal	~5 s	RxNav name checks; referral only
Exact-question cache hit	~0.01–0.02 s	Returns cached answer
Direct LLM query	~0.7–0.8 s	No retrieval branch
Medical query with RAG hit	~7–11 s	Retrieval, synthesis and verification

3.2 Implemented controls and boundaries

Safety behavior is implemented through pattern-based crisis/emergency screening, refusals for paediatric/pregnancy dosing and interaction clearance, removal of unsupported dosage numbers, post-generation diagnosis verification, and a metadata-only audit/review queue. Conversation history is held in SQLite; semantic recall uses a session-filtered Chroma collection. Exact-match in-memory TTL caching and request rate limiting are also configured. [1, 4]

The project explicitly does not diagnose. Its README also states that crisis matching may miss cases, drug interaction support is limited to name recognition, and verification risk thresholds are unvalidated heuristics. There is no evidence here of a clinical validation study; its latency figures should not be presented as proof of medical accuracy. [1]

3.3 Milestone 3 outcome

The repository provides a traceable connection between the architecture, named modules, configured medical-PDF data source, vector-retrieval baseline, and API-level answer flow. The documented baseline establishes what pipeline is built and the limited timing observations available; it does not substantiate supervised model-training results or clinical performance claims. [1–8]

Repository evidence

[1] GitHub project README — github.com/nidhi752/AI-in-Healthcare/blob/main/README.md

[2] `backend/app/core/langgraph_workflow.py`

[3] `backend/app/main.py`

[4] `backend/app/services/chat_service.py`

[5] `backend/app/agents/parallel_retrieval_agent.py`

[6] `backend/app/tools/pdf_loader.py` and `backend/app/tools/vector_store.py`

[7] `backend/app/agents/executor.py` and `backend/app/core/safety_router.py`

[8] `backend/app/tools/model_gateway.py`